

Мини-движок игры

Сюжет

Ты просыпаешься в темнице без имени. Без прошлого. Только влажный камень под спиной и далёкий гул, от которого дрожат стены.

Когда-то давно маг по имени **Балин** заключил сделку с существами из другого мира. Он обещал им проход - взамен на силу, которая позволила бы ему захватить королевство. Он получил что хотел. Королевство пало за одну ночь. Города горели. Те кто выжил говорили что небо той ночью было красным - не от огня, а от чего-то другого, чему не было имени.

Но существа не ушли. Они ждут у врат, которые Балин выстроил в сердце своей крепости. С каждой ночью стены между мирами истончаются. Твари просачиваются сквозь камень - сначала тени, потом нечто хуже. Именно они бродят по коридорам этой крепости уже двести лет.

Балин мёртв. Но его крепость стоит. И врата - тоже.

Старый маг **Валериус** открывает твою камеру в ночи. Он не объясняет как попал сюда. Говорит только одно: врата Балина заперты, и чтобы пройти через них нужно найти два артефакта - *Душевный камень* в башне стражи и *Древний свиток* в часовне. Вместе они откроют проход. За вратами находится древняя церковь, а в ней - дух священника по имени **Туримар**. Только он хранит *Печать Балина* - единственное что способно навсегда закрыть разлом между мирами.

В башне стражи Душевный камень лежит в запертом сундуке. Ключ при стражнике. В часовне Древний свиток хранится на алтаре, но просто так его не взять: у входа ловушка, и забрать свиток можно только если дойти к алтарю без единой царапины. Тогда бард **Алемет** - призрак что сторожит алтарь - поймёт что перед ним настоящий воин и отдаст свиток сам.

Когда оба артефакта собраны, врата Балина откроются. За ними - церковь. В церкви - Туримар и Печать.

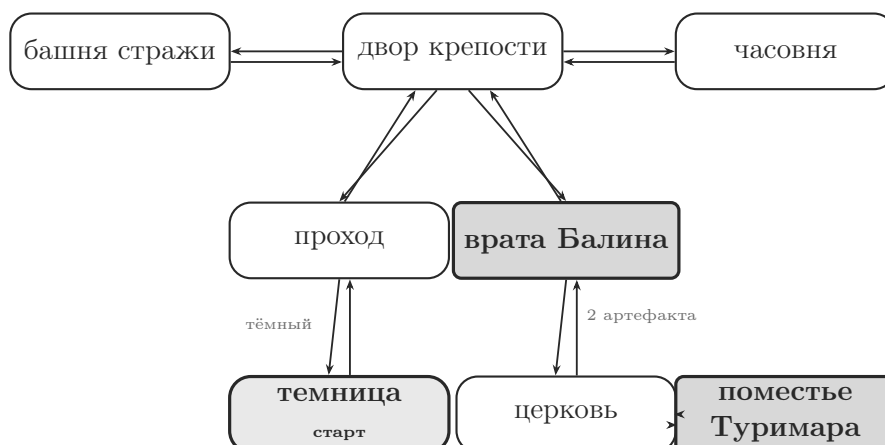
Валериус исчезает прежде чем ты успеваешь спросить зачем ему нужен именно ты. Ответ найдёшь сам - если доживёшь.

Концовки

Концовка 1 - Печать. Туримар отдаёт Печать Балина. Ты возвращаешься к вратам и проводишь ритуал. Земля вздрогнула. Врата схлопнулись - тихо, почти разочаровывающе. Утро оказалось холодным и серым. Валериус не пришёл. На камне у выхода лежала его мантия и записка без единого слова.

Концовка 4 - Правда о Валериусе (секретная). Выживший солдат в башне знает больше чем говорит. В его записях - имя. Валериус не охотник за нечистью. Он последний ученик Балина. Печать которую он просил тебя найти не закроет врата - она откроет их полностью. Туримар подтверждает это если спросить его напрямую. Ты уничтожаешь Печать прежде чем Валериус успевает остановить тебя. Он смотрит на тебя долго. Потом уходит в темноту без слов. *«Иногда спаситель и есть угроза.»*

Схема локаций



Описание локаций

Темница. Стартовая точка. Сырые стены, ржавые решётки. В сундуке у стены лежит ключ от выхода. Монстров нет.

Проход. Узкий тёмный тоннель. Без источника света наносит 10 урона каждый ход. 1 монстр - теневой страж, слабый.

Двор крепости. Центральный хаб. Открытое пространство с потухшими кострами. Здесь стоит Валериус - единственный NPC во дворе. 1–2 монстра средней силы.

Башня стражи. Дверь заперта на ключ стражника. Внутри: выживший солдат, тело стражника с ключом от сундука, сундук с Душевым камнем. 1 монстр.

Часовня. Ловушка у входа. Алемет на алтаре отдаёт Древний свиток только если игрок пришёл к нему без урона. 1–2 монстра.

Врата Балина. Открываются только при наличии обоих артефактов одновременно - Душевного камня и Древнего свитка. За ними проход в церковь. 2 сильных монстра охраняют подход.

Церковь. Подземная церковь за воротами. Здесь живёт **Туримар** - дух бывшего священника, хранитель Печати Балина. Знает полную историю Балина и Валериуса. 1–2 монстра высокой силы.

Поместье Туримара. Руины старого поместья при церкви. Записи о прошлом, скрытое золото. 2 сильных монстра.

Классы персонажей

Каждый персонаж начинает с тремя зельями здоровья. Каждое зелье восстанавливает 70% от максимального НР. При переходе в новую локацию если зелий меньше трёх их количество восстанавливается до трёх. Золото получается только с убитых монстров - каждый монстр даёт 20 монет. В начальной локации монстров нет, в остальных по 1–2, и чем дальше локация от старта тем выше их здоровье и урон.

Рыцарь

НР: 130. Золото: 100. Сила: 8, Интеллект: 2, Вера: 3, Ловкость: 3. Оружие Longsword, базовый урон 25, итоговый $25 \times (1 + 8 \times 0,05) = 35$. Зелье восстанавливает $130 \times 0,7 = 91$ НР. Ломает решётки и баррикады силой. Получает на 5 урона меньше от ловушек благодаря броне.

Маг

НР: 70. Золото: 75. Сила: 2, Интеллект: 9, Вера: 4, Ловкость: 3. Оружие Staff с атакой на расстоянии - маг не получает ответный урон от монстра в ближнем бою. Базовый урон 15, итоговый $15 \times 1,45 = 22$. Начинает с Душевым камнем в инвентаре. Не получает урон в тёмном проходе. Зелье восстанавливает $70 \times 0,7 = 49$ НР.

Вор

НР: 90. Золото: 125. Сила: 4, Интеллект: 4, Вера: 2, Ловкость: 9. Оружие Dagger, базовый урон 20, итоговый $20 \times 1,45 = 29$, первый удар удваивает урон до 58. Открывает замки без ключа. При Ловкости выше 7 проходит ловушки без урона. Зелье восстанавливает $90 \times 0,7 = 63$ НР.

Клирик

НР: 100. Золото: 75. Сила: 5, Интеллект: 3, Вера: 9, Ловкость: 3. Оружие Маса, базовый урон 18, итоговый $18 \times 1,45 = 26$. Колокольчик Bell активируется командой прау раз в 3 хода и восстанавливает $20 \times 1,45 = 29$ НР дополнительно к зельям. Нежить в церкви и поместье получает от клирика двойной урон. Зелье восстанавливает $100 \times 0,7 = 70$ НР.

Формула урона: базовый $\times (1 + \text{параметр} \times 0,05)$. Стоимость прокачки параметра: $50 + \text{уровень_прокачки} \times 25$.

Интерфейсы

ICondition. Это общий язык для всего что умеет проверять состояние игры. Когда дверь хочет понять есть ли у игрока ключ, когда событие хочет понять достаточно ли у него здоровья, когда квест хочет понять выполнен ли он - все они обращаются к одному методу `Check()`. Это позволяет передавать любое условие в любое место не зная что именно там проверяется. `HasItemCondition`, `FlagCondition`, `AndCondition` - всё это разные условия, но для двери они одинаковы: дай мне ответ да или нет. Метод: `bool Check(GameState state)`.

IEffect. Это общий язык для всего что умеет изменять состояние игры. Когда сундук открывается - он применяет список эффектов. Когда событие срабатывает - оно применяет список эффектов. Сам сундук не знает будет ли там урон, лечение или открытие двери - он просто вызывает `Apply()` у каждого эффекта по очереди. Это позволяет собирать любое поведение из готовых кубиков не трогая сам объект. Метод: `void Apply(GameState state)`.

IInteractable. Это общий язык для всего с чем игрок может взаимодействовать. Команда `interact` получает `id` объекта, ищет его в локации и вызывает `Interact()` - не зная сундук это, дверь, ловушка или NPC. Каждый объект сам решает что произойдёт. Это означает что добавление нового типа объекта никак не затрагивает саму команду. Поля и методы: `string Id`, `string Interact(GameState state)`.

ICommand. Это общий язык для всех команд игрока. `Game` хранит все команды в одном словаре. Когда игрок вводит текст - `Game` находит нужную команду и вызывает `Execute()` не зная что именно та делает внутри. Это означает что добавление новой команды не требует изменений в `Game` - достаточно добавить её в словарь. Поля и методы: `string Name`, `string Description`, `string Execute(string[] args, GameState state)`.

Абстрактные классы

ConditionBase : ICondition. Все условия в игре устроены одинаково с точки зрения хранения - у каждого есть описание которое можно вывести в интерфейсе. Это описание объявлено здесь один раз и наследуется всеми дочерними классами. Но то как именно проверяется условие у каждого своё - `HasItemCondition` смотрит в инвентарь, `FlagCondition` смотрит в словарь флагов, `HealthCondition` сравнивает числа. Поэтому метод `Check()` здесь не реализован - он только объявлен как абстрактный и каждый дочерний класс обязан написать его сам. Поля: `string Description`. Абстрактный метод: `bool Check(GameState state)`.

EffectBase : IEffect. Та же история что и с условиями. Все эффекты хранят описание - один раз здесь. Но `Apply()` у каждого своя логика: `DamageEffect` снимает HP, `AddItemEffect` кладёт предмет в инвентарь, `ChangeLocationEffect` меняет текущую локацию. Базовый класс не знает что именно нужно сделать - он только обязывает дочерние классы реализовать это самим. Поля: `string Description`. Абстрактный метод: `void Apply(GameState state)`.

GameEventBase. Все события в игре работают по одной схеме: есть условие, есть список эффектов, есть флаг одноразовости. Метод `TryTrigger` полностью реализован здесь - он проверяет условие и если оно выполнено применяет все эффекты по очереди, а потом при необходимости помечает событие как уже сработавшее. Это одинаково для всех типов событий. Но вот когда именно проверять событие - это у каждого типа своё. `OnEnterLocationEvent` проверяется при входе, `OnTurnEvent` каждый ход, `OneTimeEvent` только пока не сработало. Именно это решение и выносится в абстрактный метод `ShouldCheck()`. Поля: `ICondition Condition`, `List<IEffect> Effects`, `bool IsOneTime`, `bool WasTriggered`. Реализованный метод: `void TryTrigger(GameState state)`. Абстрактный метод: `bool ShouldCheck(GameState state)`.

CommandBase : ICommand. Все команды имеют имя и описание которое выводит `HelpCommand`. Эти поля объявлены здесь один раз. Но что именно делает команда при вызове - это уникально для каждой. `LookCommand` описывает локацию, `GoCommand` перемещает, `AttackCommand` запускает бой. Поэтому `Execute()` абстрактный. Поля: `string Name`, `string Description`. Абстрактный метод: `string Execute(string[] args, GameState state)`.

Основные классы

GameState. Это единственное место в программе где хранится всё состояние игры. Ни один класс не меняет мир напрямую - все получают GameState как параметр и работают только через его методы. Это важно потому что если в будущем нужно добавить логирование, сохранение или отмену действий - менять нужно будет только одно место. Хранит: `int Health, MaxHealth, Gold, Turn, PotionCount, List<string> Inventory, Dictionary<string, bool> Flags, List<string> EventLog, string CurrentLocation, bool GameOver, List<Quest> Quests`. Методы: `AddItem, RemoveItem, HasItem, SetFlag, GetFlag, Damage, Heal, AddGold, RemoveGold, Log, NextTurn, UsePotion`.

Player. Хранит всё что относится к конкретному персонажу - класс, параметры, урон, лечение. Метод `ApplyClass` устанавливает начальные значения при старте игры в зависимости от выбранного класса. Метод `RecalculateStats` пересчитывает `WeaponDamage` и `HealAmount` каждый раз когда параметр прокачивается - чтобы не пересчитывать каждый удар во время боя. Метод `GetUpgradeCost` возвращает стоимость следующего улучшения по формуле $50 + \text{level} \times 25$. Поля: `PlayerClass Class` (enum: `Warrior, Mage, Thief, Cleric`), `int Strength, Intelligence, Faith, Agility, WeaponDamage, HealAmount`.

Location. Одна локация мира. Хранит всё что в ней есть: название, описание, объекты с которыми можно взаимодействовать, события которые в ней срабатывают, выходы в другие локации и список монстров. Метод `ProcessEvent` перебирает все события и вызывает `TryTrigger` у каждого - это делается при входе и каждый ход. Метод `GetObject` ищет объект по `id` чтобы команда `interact` могла его найти. Метод `AddExit` добавляет новый выход динамически - именно так открываются врата Балина когда собраны артефакты. Поля: `string Id, Name, Description, List<IInteractable> Objects, List<GameEventBase> Events, Dictionary<string, string> Exits, List<Monster> Monsters`.

Game. Главный класс который держит всё вместе. Хранит состояние игры, словарь всех локаций, словарь всех команд и текущего игрока. Метод `Start` показывает экран выбора класса, вызывает `BuildWorld` для создания всего мира и запускает игровой цикл. Метод `RunLoop` читает ввод от игрока пока `GameOver` не станет `true`. Метод `ProcessInput` разбирает ввод через `switch`, находит нужную команду и вызывает `Execute`. Метод `BuildWorld` создаёт все локации, расставляет в них объекты события и монстров и связывает всё вместе. Поля: `GameState State, Dictionary<string, Location> Locations, Dictionary<string, ICommand> Commands, Player CurrentPlayer`.

Quest. Задание которое само следит за своим выполнением. Хранит название, описание, флаг завершения и условие завершения. Метод `Update` вызывается каждый ход - он проверяет условие и если оно выполнено ставит `IsCompleted` в `true`. Это означает что квест не нужно вручную завершать из разных мест кода - он сам поймёт когда выполнен. Поля: `string Name, Description, bool IsCompleted, ICondition CompletionCondition`.

Monster. Противник привязанный к локации. Хранит имя, здоровье, урон и награду за убийство - всегда 20 монет. После смерти помечается как мёртвый и больше не участвует в бою. Монстры создаются в `BuildWorld`: чем дальше локация от старта тем выше `Health` и `Damage`. Поля: `string Name, int Health, Damage, GoldReward, bool IsDead`. Метод: `bool IsAlive()`.

PotionSystem. Отвечает за всю логику зелий. Метод `OnLocationEnter` вызывается каждый раз когда игрок входит в новую локацию - если зелий меньше трёх восстанавливает до трёх. Метод `UsePotion` списывает одно зелье и восстанавливает $\text{MaxHealth} \times 0.7$ HP округлённо. Вынесен в отдельный класс чтобы не загромождать `GameState` или `GoCommand`.

Команды

LookCommand выводит название и описание текущей локации, список объектов с которыми можно взаимодействовать, список выходов и живых монстров если они есть.

GoCommand принимает направление, проверяет наличие выхода в текущей локации, меняет CurrentLocationId, вызывает ProcessEvents для новой локации и PotionSystem.OnLocat для восстановления зелий.

InteractCommand принимает id объекта, ищет его в текущей локации через GetObject и вызывает Interact. Если объект не найден - сообщает об этом.

AttackCommand принимает имя монстра, наносит WeaponDamage урона монстру, монстр отвечает своим Damage если ещё жив. Если монстр погибает - вызывается AddGoldEffect(20) и пишется сообщение в лог.

InventoryCommand выводит список предметов в инвентаре, количество золота и количество оставшихся зелий.

StatusCommand выводит HP, максимальное HP, класс, урон, все параметры с уровнями прокачки, количество зелий, номер хода и статус квестов.

HelpCommand перебирает словарь команд и выводит имя и описание каждой.

UpgradeCommand принимает название параметра, проверяет хватает ли золота через GoldCondition, вызывает UpgradeStatEffect и RemoveGoldEffect, затем RecalculateStats для пересчёта урона.

DrinkCommand вызывает PotionSystem.UsePotion. Если зелий нет - сообщает об этом.

PrayCommand только для клирика. Проверяет флаг isCleric и счётчик healCooldown. Если прошло не менее 3 ходов - применяет HealEffect(HealAmount) и сбрасывает счётчик.

Условия

HasItemCondition возвращает true если предмет с заданным id есть в инвентаре. Поля: string ItemId.

FlagCondition возвращает true если флаг мира совпадает с ожидаемым значением. Поля: string FlagName, bool Expected.

HealthCondition возвращает true если здоровье игрока выше или ниже порогового значения. Поля: int Threshold, bool LessThan.

GoldCondition возвращает true если у игрока достаточно золота для операции. Поля: int RequiredAmount.

NoDamageCondition возвращает true если здоровье игрока равно максимальному, то есть урона не получено. Используется Алеметом в часовне для проверки что игрок дошёл до алтаря чистым.

AndCondition возвращает true только если все вложенные условия вернули true. Поля: List<ICondition> Conditions.

OrCondition возвращает true если хотя бы одно вложенное условие вернуло true. Поля: List<ICondition> Conditions.

NotCondition инвертирует результат другого условия. Поля: ICondition Inner.

Эффекты

AddItemEffect добавляет предмет в инвентарь и пишет в лог. Поля: string ItemId.

RemoveItemEffect удаляет предмет из инвентаря. Поля: string ItemId.

SetFlagEffect устанавливает флаг мира в заданное значение. Поля: string FlagName, bool Value.

DamageEffect уменьшает здоровье игрока. Поля: int Amount.

HealEffect восстанавливает здоровье игрока не превышая максимум. Поля: int Amount.

LogEffect записывает сообщение в журнал событий. Поля: string Message.

AddExitEffect добавляет новый выход в указанную локацию - именно этот эффект открывает врата Балина когда собраны оба артефакта. Поля: string LocationId, string Direction, string TargetId.

ChangeLocationEffect меняет текущую локацию игрока. Поля: string TargetLocationId.

AddGoldEffect добавляет золото в GameState. Поля: int Amount.

RemoveGoldEffect списывает золото из GameState. Поля: int Amount.

UpgradeStatEffect увеличивает параметр игрока на 1 и вызывает RecalculateStats. Поля: string Stat.

EndGameEffect записывает финальное сообщение и ставит GameOver в true. Поля: string Message.

Объекты взаимодействия

Chest. Контейнер который выдаёт предметы при выполнении условия. Если уже открыт - сообщает об этом. Иначе проверяет `OpenCondition`, применяет все эффекты из списка по очереди и помечает себя как открытый. Пример: сундук с Душевым камнем требует флаг `towerDoorOpen` и наличие ключа стражника. Поля: `string Id`, `ICondition OpenCondition`, `List<IEffect> Effects`, `bool IsOpened`.

Door. Дверь которая открывает проход при выполнении условия. При успехе вызывает `AddExitEffect` добавляя новый выход. При неудаче сообщает что заперто. Врата Балина реализованы как `Door` с условием `AndCondition` по обоим артефактам. Поля: `string Id`, `ICondition OpenCondition`, `List<IEffect> Effects`, `bool IsOpened`.

NPC. Персонаж с диалогами зависящими от состояния игры. При взаимодействии перебирает список пар условие-фраза по порядку и выводит первую у которой `Check()` вернул `true`. Валериус: нет артефактов - подсказывает где искать, есть оба - говорит идти к вратам, игрок знает правду - молчит. Туримар: хранит Печать и историю Балина, при правильном вопросе раскрывает тайну Валериуса. Алемет: проверяет `NoDamageCondition` и если выполнено передаёт Древний свиток. Поля: `string Id`, `List<(ICondition, string)> Dialogues`.

Trap. Ловушка которая срабатывает один раз. После срабатывания помечает себя и больше не активируется. Ловушка у входа в часовню - именно она проверяет получит ли игрок урон перед встречей с Алеметом. Поля: `string Id`, `List<IEffect> Effects`, `bool IsTriggered`.

Merchant. Торговец у которого можно прокачивать параметры за золото. При взаимодействии выводит список доступных параметров, их текущие уровни и стоимость следующего улучшения. Поля: `string Id`, `List<string> AvailableStats`.

События

OnEnterLocationEvent. `ShouldCheck` возвращает `true` при каждом входе в локацию. Пример: атмосферное описание при входе в церковь, каждый раз.

OnTurnEvent. `ShouldCheck` возвращает `true` каждый ход пока игрок в локации. Пример: тёмный проход проверяет `AndCondition` из двух `NotCondition` - нет факела и не маг - и если оба выполнены наносит 10 урона.

OneTimeEvent. `ShouldCheck` возвращает `true` только пока `WasTriggered` равно `false`. После первого срабатывания не проверяется снова. Пример: Валериус произносит вступление при первом выходе во двор.

Квесты

Квест 1 - «Побег из темницы». Цель добраться до двора крепости. `CompletionCondition`: `FlagCondition("courtyardReached")`. Флаг ставится через `SetFlagEffect` в `OneTimeEvent` при первом входе во двор.

Квест 2 - «Запечатать врата Балина». Цель найти оба артефакта, открыть врата, получить Печать у Туримара и провести ритуал. `CompletionCondition`: `AndCondition(HasItemC`

HasItemCondition("ancientScroll"), HasItemCondition("balinSeal"), FlagCondition("ritu